Designing a Modular and Maintainable HealthCare Appointment Management System

Tools Allowed	Duration & Team Size
• UML tools: StarUML, Draw.io, etc. • IDEs (for prototypes): IntelliJ, Eclipse, VS Code, etc.	• **Project submission deadline** : May 24, 2026 • **Team Size**: 5 Student

Objective:

Software design and modeling using Object-Oriented principles, applying **SOLID** and **GRASP** principles and incorporating suitable **design patterns**.
The primary objective of this project is not to deliver a fully implemented application, but to focus on **software design quality**, modularity, and extensibility.
This project simulates a real-world software modeling task requiring **design thinking and best practices** in object-oriented software engineering.
Your design must ensure:

- Easy adaptability to new features
- Clean separation of concerns
- Low coupling and high cohesion
- Code that is easily extendable and testable

Background

A healthcare startup is planning to launch a **digital platform for managing medical appointments** between patients, doctors, and clinics. The platform is expected to grow rapidly and support different medical specialties, clinics, and services over time.

You have been hired as part of a **Software Design Team** to propose the **core system design**. Your mission is to deliver a structured, object-oriented design that follows recognized software engineering principles and patterns.

Your design should ensure:

- Easy adaptability to new features
- Clean separation of concerns
- Low coupling and high cohesion
- Code that can be easily extended and tested

Functional Overview: E-Commerce Platform

The healthcare appointment management system is intended to provide a structured and centralized digital solution facilitating interactions among patients, medical staff, and administrative personnel. The system is designed to support the discovery of healthcare services, the scheduling and management of medical appointments, and the coordination of availability and communication among all stakeholders. Emphasis is placed on role-based access, functional separation, and consistency of operations, ensuring that each category of user interacts with the system according to clearly defined responsibilities and permissions.

User management constitutes a core functional component of the system. The platform must enable users to register securely, authenticate through reliable identification mechanisms, and manage personal profile information. Access to system functionalities is governed by user roles, thereby enforcing authorization rules and ensuring system security. This role-based approach guarantees that users can only access features relevant to their responsibilities and prevents unauthorized interactions with sensitive system elements.

Patients represent the primary end users of the system. The platform must allow patients to browse healthcare services, consult medical clinic offerings, and schedule appointments based on availability constraints. Patients must be able to view their current and historical appointments, including completed and cancelled visits, in order to maintain transparency regarding their medical engagements. Furthermore, the system must enable patients to update personal information while ensuring data integrity and confidentiality.

Doctors act as healthcare service providers within the platform. The system must allow medical professionals to manage their availability calendars by defining working hours and blocking unavailable periods. Doctors must be able to consult their scheduled appointments through daily and weekly agenda views, ensuring effective planning of consultations. Any updates to availability must be reflected promptly within the system to maintain alignment between scheduling data and actual practitioner constraints.

Administrators are responsible for supervising and maintaining the overall operation of the system. Their role includes managing user accounts, assigning and modifying roles, overseeing clinics and healthcare services, and controlling system configuration parameters. Administrators ensure consistency, integrity, and proper coordination across the different system modules, acting as facilitators and controllers of the platform’s core processes.

The clinic and service catalog serves as an organized repository of healthcare offerings available through the system. Each medical service must be associated with a specific clinic and described using key attributes such as service name, detailed description, medical specialty, consultation duration, fee, and assigned medical professionals. The catalog must provide structured access to this information, allowing patients to browse and filter services according to predefined criteria such as specialty, geographic location, or availability. This component constitutes the foundational layer upon which appointment scheduling functionality is built.

Appointment scheduling represents one of the most critical functional aspects of the system. Patients must be able to request, modify, or cancel appointments through an interface that

enforces availability and consistency constraints. The system must verify doctor availability prior to confirming an appointment and prevent scheduling conflicts, including overlapping bookings. Each appointment follows a defined lifecycle, progressing through states such as scheduled, confirmed, completed, or cancelled, thereby ensuring traceability and accurate status representation for all users.

The system incorporates a billing and payment simulation mechanism designed to reflect real-world healthcare payment scenarios. During the appointment booking process, patients must be given the opportunity to select a preferred payment method, such as credit card, insurance coverage, or digital wallet. Although actual transaction processing is outside the scope of the platform, the system must record the selected payment option and simulate payment confirmation, ensuring consistency with real billing workflows.

In addition, the system must support pricing adjustments to accommodate insurance policies and promotional strategies. Pricing variations may include insurance coverage percentages, promotional discounts, or fixed and percentage-based reductions. These adjustments may be applied at the appointment level or across selected services. The system must compute and present the final cost accurately prior to appointment confirmation, ensuring transparency and correctness in pricing information.

User communication is managed through a notification subsystem that ensures timely dissemination of critical information. The system must generate notifications for events such as appointment confirmations, cancellations, reminders, and schedule modifications initiated by doctors. Notifications may be delivered through multiple channels, including in-application messages, emails, or short message services, depending on user preferences and system configuration. This mechanism is essential for reducing missed appointments and improving coordination among users.

Doctor availability management is a key component for maintaining scheduling reliability. Medical professionals must be able to define and update their availability patterns dynamically. The system must enforce consistency rules to prevent appointments from being scheduled during unavailable periods and to ensure that existing appointments remain valid following schedule updates. This functionality guarantees the integrity of the scheduling process and supports conflict-free appointment management.

Tasks requested
1. Use **UML diagrams** to communicate Design. 2. Apply **SOLID** principles in class structure. 3. Apply **GRASP** principles 4. Use Design **Patterns**, such as Strategy Factory, Observer, Singleton 5. Prioritize **extensibility**

Deliverables

Your project deliverables will be evaluated based on the completeness, clarity, and quality of your design artifacts and presentations. You are expected to submit a **written report**, deliver a **presentation**, and demonstrate a **prototype or code snippets** illustrating your key design ideas.

In the written report, clearly document the functional requirements, your UML diagrams (Use Case, Class, Sequence), and provide detailed explanations of how your design applies core object-oriented principles like SOLID and GRASP. You must identify and explain the use of at least three design patterns in your system.

During the presentation, every team will have 15 minutes to explain the design decisions, walk through the main system components, and justify the choices with respect to software design principles. The presentation should reflect teamwork, clarity of communication, and understanding of object-oriented design.

The prototype or code demonstration does not need to be a full implementation but should highlight key classes and interactions, showing that your design is feasible and maintainable.

1. **Written Report** (PDF/Word 10 pages max):
    - Problem statement and Functional requirements
    - UML Diagrams : use case Diagram, class Diagram (UML) and sequence diagrams (at least 3 principal interactions)
    - Explanation of how SOLID and GRASP principles were applied
    - Description of applied Design Patterns
    - Challenges faced and team reflections
2. **Presentation** (5 slides max):
    - Summary of design choices
    - Diagrams walkthrough
    - Justification of pattern and principle usage
    - Short Q&A
3. **Code Prototype in java** :
    - Simulate core classes (e.g., Appointment, Doctor, Patient, Payment, Notification)
    - Demonstration of key interactions (no full implementation required)

Evaluation Criteria

The final grade will be split as follows:

- **50% Group Evaluation** (shared deliverables)
- **50% Individual Evaluation** (written and oral technical assessment)

This assessment will evaluate each student's **understanding of the design**, contribution, and ability to justify architectural decisions.

* The Technical Evaluation will take place on **May 26**, 2026, followed by the Final Evaluation on **June 2,** 2026.